NLC STEM Club

Northeast Lakeview College (Alamo Colleges)

Fixed-Wing UAV Design and Environmental Data Acquisition Using Additive Manufacturing

---

Lead Researchers

Aiden Ware Bosanko · Kaleb Harris · Jorge Cruz
William Van Cleave · Isaiah Jimenez · Angelo Mejia

Faculty Advisors

Prof. Mitchell · Prof. Dan Ball · Prof. Sue Stankus · Prof. Rebecca Guevara

Northeast Lakeview College (Alamo Colleges)
September 11, 2025 — March 2026

Table of Contents

Abstract 1
1. Introduction 1
1.1. The Growing Role of UAVs in Environmental Science 1
1.2. From Quadcopter to Fixed-Wing: A Platform Evolution 2
1.3. Research Objectives and Scope 2
2. Design and Methodology 2
2.1. Airframe Selection and Fabrication 2
2.2. Propulsion and Control Surface Systems 3
2.3. Flight Controller and Navigation 3
2.4. Environmental Sensor Payload and Data Acquisition 3
2.5. Team Structure and Division of Responsibilities 4
2.6. Predicted Flight Parameters 4
2.7. Lessons Learned During Development 5
3. Results and Data Analysis ..... TBD
4. Discussion 5
4.1. Sources of Error and Uncertainty 5
4.2. Interpretation of Results 6
5. Advanced Applications and Future Work 6
5.1. Towards Longer Endurance and Solar-Assisted Flight 6
5.2. Expanded Sensor Suite and Real-Time Telemetry 6
5.3. Autonomous Mission Planning and AI-Driven Analysis 7
5.4. Platform Replication and Open-Source Documentation 7
6. Conclusion 7
Appendix A: Parts & Budget 8
Appendix B: Project Timeline 9
Appendix B2: Team Roles & Responsibilities 10
Appendix C: System Wiring Diagram 11
Appendix D: Photos 12
Appendix E: Arduino Sensor Logger Code 13

Fixed-Wing UAV Design and Environmental Data Acquisition Using Additive Manufacturing

Author(s)/Researchers:

- **Lead Researchers/Principal Investigators:** Aiden Ware Bosanko (Mechanical), Kaleb Harris (Mechanical & Software), Jorge Cruz (Mechanical, Software & Research Lead), William Van Cleave (Mechanical), Isaiah Jimenez (Electrical), Angelo Mejia (Electrical)
- **Researcher Contributors/Collaborators:** Samuel Cedeno, Kyle McCracken, Alex (Electrical), Austin (Electrical), Tyler Rankins (Electrical)
- **Support/Technical Contributors:** Navora Medina (Historian), Talan (Software), Chris (Software)
- **Faculty Advisors:** Professor Mitchell, Professor Dan Ball, Professor Sue Stankus, Professor Rebecca Guevara

Institution: Northeast Lakeview College (Alamo Colleges)

Date: September 11, 2025

Abstract

Introduction: Air quality in the San Antonio region is a documented public health concern, yet spatially distributed, low-altitude atmospheric data remains scarce. The NLC STEM Club at Northeast Lakeview College set out to change that by designing, building, and flying a 3D-printed fixed-wing UAV capable of collecting real-time air quality and atmospheric data across the greater San Antonio area.

Methodology: We selected the open-source Eclipson Model T airframe and fabricated it using Lightweight PLA (LW-PLA) filament, reinforced with carbon fiber wing spars. A Matek F405-WMN flight controller running INAV firmware handles all autonomous and manual flight operations. We mounted a separate Arduino Nano microcontroller to manage the sensor payload — a BMP280 atmospheric pressure sensor and an MQ135 air quality sensor — logging all readings to a SanDisk 32GB microSD card every two seconds.

Results: Physical assembly of the integrated system began March 22, 2026, following 28 weeks of design, procurement, and fabrication work. Flight testing and sensor data collection are scheduled to follow. Results — including GPS-logged flight tracks, air quality readings, and pressure-altitude profiles — will be documented and added to this report upon completion.

Conclusion: We show that a college STEM club can design, build, and fly a capable research-grade UAV using accessible components and open-source tools for under $500. Our dual-subsystem architecture — separating flight avionics from the sensor payload — is practical, reproducible, and extensible for future environmental research programs.

1. Introduction

1.1. The Growing Role of UAVs in Environmental Science

UAVs have become one of the most powerful tools available for environmental monitoring. They can reach remote or hazardous locations, fly at low altitudes, and carry modular sensor payloads — all at a fraction of the cost of manned aircraft. Traditional ground-based monitoring stations do important work, but they're stuck in one place. UAVs break that limitation by enabling dynamic, geographically distributed sampling missions that teams can deploy and reconfigure quickly.

The urgency of this research is underscored by local conditions. According to the American Lung Association's 2025 State of the Air report, the San Antonio–New Braunfels–Kerrville metro area ranked 20th worst in the nation for ground-level ozone pollution, with Bexar County averaging 10.7 unhealthy ozone days per year — earning a failing grade. Contributing factors include vehicle emissions, proximity to the Eagle Ford Shale oil and gas development, industrial activity, and the region's valley geography, which traps pollutants during temperature inversions. Fixed-wing UAVs offer a uniquely powerful tool for studying these conditions, enabling spatially distributed, low-altitude sampling across the neighborhoods, corridors, and open areas that fixed ground stations cannot reach.

The NLC STEM Club built this UAV to collect real-time environmental data — air quality, atmospheric pressure, temperature, and humidity — over the San Antonio area. We deliberately chose a fixed-wing platform over a multirotor because fixed-wing aircraft are more aerodynamically efficient and cover more ground on the same battery. Using the Eclipson Model T as our base airframe and building a custom electronics and sensor stack on top of it, our goal was a capable, reproducible, well-documented research platform that other student teams could replicate.

1.2. From Quadcopter to Fixed-Wing: A Platform Evolution

We originally planned to build a quadcopter. After a series of team discussions and a hard look at our mission requirements, we switched to fixed-wing. Quadcopters are easy to build and can hover, but they burn through battery life fast just staying in the air. For a mission focused on covering ground and sampling air quality across a wide area, that's a serious limitation. Fixed-wing aircraft glide efficiently and stay airborne much longer on the same power. The switch meant more work for our Mechanical team, who had to redesign and reprint the entire airframe — but the performance gains are worth it.

1.3. Research Objectives and Scope

The primary objectives of this project are:

- To design, fabricate, and assemble a fully functional 3D-printed fixed-wing UAV using the Eclipson Model T airframe as the structural foundation.
- To integrate a complete electronics stack including a flight controller, GPS module, ESC, brushless motor, servos, and an RC link capable of autonomous and manual flight.
- To develop and deploy an onboard environmental sensor payload — including atmospheric pressure and air quality sensors — managed by an Arduino Nano microcontroller.
- To log all collected environmental data to onboard storage for post-flight analysis and reporting.
- To document the full design, build, integration, and testing process in a formal research report suitable for academic and portfolio use.

1.4. System Overview

At a high level, the UAV system operates as two independent but physically integrated subsystems. The first is the flight-critical avionics stack: a Matek F405-WMN flight controller running INAV firmware, fed by a BN-220 GPS module and commanded via a HappyModel ELRS receiver linked to a Jumper Smart ELRS ground transmitter. This stack controls all flight surfaces through four metal gear servos and drives the SunnySky X2212 1250KV brushless motor through a 40A ESC. Power for the entire avionics stack is drawn from an OVONIC 4S 2200mAh LiPo battery. The second subsystem is the research payload: an Arduino Nano V3.0 microcontroller that independently reads from a BMP280 atmospheric pressure sensor and an MQ135 air quality sensor every two seconds, time-stamps each reading, and writes the data to a SanDisk 32GB microSD card in CSV format. A manual SPST toggle switch allows the pilot to start and stop data logging during flight without interrupting vehicle operation. This dual-subsystem architecture ensures that sensor activity cannot interfere with flight-critical avionics, and that the research payload can be developed, tested, and modified independently of the flight system.

2. Design and Methodology

2.1. Airframe Selection and Fabrication

We chose the Eclipson Model T because it's open-source, fully 3D-printable, and designed for stable, efficient flight — exactly what a club-level research build needs. We print the airframe using Lightweight PLA (LW-PLA), a thermally foamed filament that is about 40-50% less dense than standard PLA. That weight reduction makes a real difference for flight time without making the parts harder to print. We reinforce the wings with 2mm carbon fiber rods as spars, which keep the wing geometry rigid under aerodynamic loads without adding significant weight.

Our Mechanical team handles all printing, post-processing, and structural assembly. We carefully tune print parameters to get the right balance of weight and strength, and we inspect every part for layer adhesion and dimensional accuracy before it goes into the build. We bond key structural joints with epoxy and cyanoacrylate — the same approach used across professional and hobbyist RC aircraft builds.

2.2. Propulsion and Control Surface Systems

We drive the aircraft with a SunnySky X2212 1250KV brushless motor — a reliable, well-regarded choice that matches our expected all-up weight. A Gemfan nylon propeller pairs with the motor, and a 40A brushless ESC rated for 2-4S LiPo handles the power conversion. We run everything off an OVONIC 4S 2200mAh 14.8V LiPo battery, which gives us the energy density we need to hit our target flight time.

We use four RGBZONE ES08MA II metal gear servos to actuate the ailerons, elevator, and rudder. We went with metal gears specifically because they hold their neutral position more consistently than plastic gears — something that matters a lot for stable fixed-wing flight. Each servo connects to the flight controller through standard extension cables.

2.3. Flight Controller and Navigation

Our flight controller is the Matek F405-WMN running INAV firmware. We picked this combination because INAV has excellent native fixed-wing support, a clean configurator interface, and autonomous flight features that are genuinely accessible without a steep learning curve — important for a club where not everyone has flight controller experience coming in. The F405-WMN brings onboard OSD and blackbox logging on top of that. Together they give us fully manual control, GPS-stabilized flight, waypoint navigation, loiter, and return-to-home — everything we need to run a structured environmental sampling mission.

A BN-220 dual GPS module feeds position and navigation data to INAV over TTL serial, enabling position hold, heading stabilization, and autonomous waypoint missions. On the radio side, we use a Jumper Smart ELRS 2.4GHz transmitter on the ground paired with a HappyModel ELRS nano receiver onboard. ELRS gives us a low-latency, long-range control link with live telemetry feedback — well beyond what we need for our planned missions.

2.4. Environmental Sensor Payload and Data Acquisition

We manage the entire research payload with an Arduino Nano V3.0 that runs completely independently from the flight controller. We deliberately separated these two systems so that sensor activity can never affect flight stability — if something goes wrong with the data logging, the plane keeps flying. The payload itself includes a BMP280 for barometric pressure and altitude, and an MQ135 for air quality monitoring, including volatile organic compounds (VOCs) and CO_2-equivalent readings.

Every sensor reading gets time-stamped and written to a SanDisk 32GB microSD card through an onboard card reader module. We added a manual SPST toggle switch to the payload circuit so the pilot or ground crew can start and stop logging mid-flight without touching the avionics. That lets us target

specific flight phases for data collection — like activating logging only when we enter a loiter pattern over a point of interest — which keeps the data clean and the post-flight analysis manageable.

Table 1: Onboard Sensor Specifications

Specification	BMP280	MQ135	Notes
Measures	Pressure, temperature, altitude	VOC, CO_2-equivalent, air quality	*Primary environmental parameters*
Interface	I2C (address 0x76)	Analog (A1 on Arduino Nano)	*BMP280 via Wire library; MQ135 via analogRead()*
Supply Voltage	3.3V	5V	*Different voltages — wiring must be separated*
Pressure Range	300–1100 hPa	N/A	*Covers sea level to ~9,000m altitude*
Temperature Range	−40°C to +85°C	−10°C to +50°C (operating)	*BMP280 has wider operating range*
Pressure Accuracy	±1 hPa (typical)	N/A (relative output)	*BMP280 factory calibrated*
Output Format	Compensated float values	10-bit ADC raw (0–1023)	*MQ135 converted to percentage in firmware*
Altitude Resolution	~0.16m (high resolution mode)	N/A	*Computed from pressure using barometric formula*
Warm-up Time	~2ms (fast startup)	24–48 hours for baseline stability	*MQ135 requires extended burn-in for accuracy*
Data Rate	Up to 157 Hz (normal mode)	Limited by Arduino analog read speed	*Logged at 0.5 Hz (every 2 seconds) in firmware*

2.5. Team Structure and Division of Responsibilities

We organized the project into four interdisciplinary teams, each led by a club officer, all working in parallel throughout the build:

- Mechanical & Structural Team (Aiden Ware Bosanko, Kaleb Harris, Jorge Cruz): Responsible for all 3D printing, airframe assembly, structural reinforcement, and physical integration of components into the fuselage and wing structures.
- Electronics & Power Team (Isaiah Jimenez, Angelo Mejia, Alex, Austin, Tyler Rankins): Responsible for all wiring, soldering, ESC configuration, power distribution, and electrical system validation.
- Software & Data Team (Kaleb Harris, Jorge Cruz, Talan, Chris): Responsible for flight controller configuration via the INAV Configurator, Arduino sensor code development, data logging pipeline, and post-flight data analysis.
- Documentation & Reporting Team: Responsible for maintaining build logs, capturing photos and video throughout the process, and compiling this final research report.

2.6. Predicted Flight Parameters

Based on the Eclipson Model T specifications and the selected component stack, the following flight parameters have been estimated prior to first flight. These values will be verified and updated following flight testing.

Parameter	Predicted Value	Notes
Airframe	Eclipson Model T	*Open-source 3D-printable fixed-wing*
Wingspan	~1,200 mm (47 in)	*Per Eclipson Model T design specs*
Est. All-Up Weight	~800–1,100 g (1.8–2.4 lbs)	*Battery, electronics, and payload included*
Cruise Speed	~45–65 km/h (28–40 mph)	*Based on motor/prop selection*
Stall Speed	~28–35 km/h (17–22 mph)	*Conservative estimate for safe hand-launch*
Expected Flight Time	~20–35 minutes	*4S 2200mAh at cruise throttle*
Flight Modes	Manual, Stabilize, GPS Hold, Waypoint	*Via INAV on Matek F405-WMN*
Data Collection Altitude	~30–100 m AGL	*Target operational altitude range*
Launch Method	Hand launch	*Belly landing on grass — no landing gear*

2.7. Lessons Learned During Development

Building this UAV over 28 weeks taught us a lot — some lessons came easy, others cost us time. The biggest was the mid-project switch from quadcopter to fixed-wing. Our core mission never changed, but the platform change meant our Mechanical team had to scrap weeks of design work and start over. Looking back, if we had done a more thorough mission analysis at the very beginning, we would have landed on fixed-wing from day one.

Parts sourcing was another hard lesson. We ended up swapping out four components mid-build — the motor, propeller, sensor package, and filament — because of availability, compatibility, or cost issues that we didn't catch early enough. Next time, we'd identify backup options for every critical component during the planning phase rather than scrambling for replacements mid-build.

INAV surprised us with how much there was to learn. Members who hadn't worked with flight controllers before found the configuration process more involved than expected. We'd strongly recommend that future teams spend dedicated time just getting comfortable with the firmware on a bench setup — before any wiring goes into the airframe.

The one decision we'd make again without hesitation: keeping the Arduino sensor payload completely separate from the flight controller. It let our Software and Electronics teams work in parallel without stepping on each other, and any sensor problem we hit could be debugged in isolation without ever touching the flight system. Clean separation of concerns makes everything easier.

4. Discussion

4.1. Sources of Error and Uncertainty

While full discussion of results is pending flight testing, several known sources of error and uncertainty have been identified during the design and build phases. These are documented here to provide context for interpreting future flight data.

Sensor Calibration: The MQ135 gas sensor requires a warm-up period of approximately 24–48 hours in clean air for accurate baseline calibration. Readings taken during early flights may exhibit elevated baseline drift until the sensor stabilizes. The BMP280 pressure sensor is factory-calibrated but may require an altitude offset correction based on local atmospheric conditions at the time of flight.

GPS Accuracy: The BN-220 GPS module has a stated horizontal position accuracy of approximately 2.5m CEP under open-sky conditions. Flights near buildings, trees, or other obstructions may experience degraded GPS accuracy, affecting the precision of geotagged sensor data.

Airspeed Variation: Without a dedicated airspeed sensor, the UAV relies on GPS ground speed for navigation. Wind conditions will introduce variation between ground speed and true airspeed, potentially affecting motor throttle levels and sensor exposure time at any given location.

Sensor Placement and Thermal Interference: Placement of the BMP280 and MQ135 sensors within or near the fuselage may expose them to heat generated by the ESC, motor, and flight controller. This thermal interference could introduce positive bias in temperature and air quality readings. Proper shielding and placement away from heat sources will be critical.

Manual Data Logging Trigger: The use of a manual SPST toggle switch to initiate and terminate data logging introduces operator-dependent variability. There may be inconsistencies in exactly when logging begins and ends relative to specific flight phases. A future improvement would be to trigger logging automatically based on flight mode or GPS altitude.

LW-PLA Structural Variability: Lightweight PLA exhibits variable expansion characteristics during printing. Differences in print temperature, ambient humidity, and layer adhesion across multiple print sessions may result in slight dimensional inconsistencies between airframe components, potentially affecting aerodynamic symmetry.

4.2. Interpretation of Results

[To be completed following flight testing. Will include analysis of sensor data, flight performance, and comparison against predicted parameters.]

5. Advanced Applications and Future Work

5.1. Towards Longer Endurance and Solar-Assisted Flight

We originally planned to include solar panels — thin-film cells mounted on the wing surfaces, with a boost/buck converter feeding charge back into the flight battery. We descoped that for the initial build to keep complexity and cost manageable, but we haven't abandoned it. The Eclipson Model T's broad, flat wing surfaces are actually ideal for solar integration. Even a modest supplemental charge during cruise could meaningfully extend mission time, especially on a clear Texas day.

A natural next step would be to bring solar back in — starting with ground-based characterization tests to measure realistic charge rates under San Antonio sunlight before committing to full integration.

5.2. Expanded Sensor Suite and Real-Time Telemetry

Right now our payload focuses on pressure and air quality. That's a solid start, but there's a lot of room to grow. A BME280 would add temperature and humidity to the mix. PM2.5/PM10 particulate sensors would open the door to real air pollution studies. A UV index sensor could map solar radiation across the city. The Arduino Nano has plenty of I2C and analog headroom to run several of these at the same time.

We're also excited about real-time telemetry. Right now everything gets analyzed after the flight, but there's no reason we couldn't stream sensor data live to the ground station via the ELRS link or a dedicated telemetry radio. Imagine watching air quality readings update on a map in real time as the plane flies overhead — and choosing to loiter over a pollution hotspot on the spot rather than discovering it in post-processing.

5.3. Autonomous Mission Planning and AI-Driven Analysis

INAV supports full waypoint missions, which means we can pre-program a grid pattern at a fixed altitude and let the aircraft execute it autonomously. That opens up systematic, repeatable sampling — fly the same grid at dawn and at noon, and you've got a direct comparison of morning versus midday air quality across the same geography. No pilot input required once the mission starts.

Further down the road, the time-stamped CSV files we log are a natural fit for machine learning. Train an algorithm on enough flight data and it could identify spatial air quality patterns, correlate readings with altitude or wind speed, or flag anomalies worth investigating. That's exactly what professional environmental UAV programs are doing right now — and it's a realistic next step for this project.

5.4. Platform Replication and Open-Source Documentation

One outcome we're genuinely proud of is how replicable this build is. The Eclipson Model T is open-source. Every component we chose is available on Amazon. The total budget is under $500. Another student team — anywhere — could reproduce this platform from our parts list alone. We plan to open-source our Arduino code, our INAV configuration files, and our full build documentation so other clubs don't have to start from scratch the way we did.

6. Conclusion

We set out to prove that a college STEM club could build a real research-grade UAV — not from a kit, not with a faculty member doing the hard parts, but through genuine engineering work spanning mechanical fabrication, electronics integration, firmware configuration, and software development. This report documents what that actually looks like: 28 weeks, one major platform pivot, four teams, and a lot of problem-solving.

Switching from quadcopter to fixed-wing was the hardest call we made. It set our Mechanical team back and forced everyone to learn new skills fast. But it was the right call. A well-configured fixed-wing UAV

covers more ground, stays airborne longer, and is simply a better tool for wide-area environmental monitoring. We'd make that call again.

What we built here — a flight-critical avionics stack running INAV, completely separated from an independent Arduino sensor payload — is modular, extensible, and designed to be built upon. This isn't a finished product. It's a foundation. The future work section lays out a real roadmap: solar endurance, live telemetry, autonomous grid missions, and eventually machine learning analysis of the data we collect.

[Final flight results and data analysis to be added following flight testing.]

What this project ultimately proves is simple: the gap between a student club and a serious aerospace research program is a lot smaller than it looks. The tools exist. The knowledge is accessible. The drive is here. We just have to show up and build it.

Acknowledgements

This project would not have been possible without the support and guidance of our faculty advisors: Professor Mitchell, Professor Dan Ball, Professor Sue Stankus, and Professor Rebecca Guevara. Their encouragement, technical input, and willingness to give us the space to learn by doing this project shaped at every stage.

We also want to recognize Navora Medina, whose work as our project historian kept the team organized and on track across 28 weeks a job that is easy to overlook and impossible to replace. And every club member who showed up, asked questions, soldered a wire, or printed a part: this is your project too.

We owe special acknowledgement to the founding members of the NLC STEM Club. Two years ago, Kaleb Harris, Jorge Cruz, Aiden Ware Bosanko, Sam Cedeno, and Thomas Goosetree built this club from nothing. Aiden served as Vice President, Sam as our Technical Lead, and Thomas as a co-founder and together they set the vision, the culture, and the standard that made a project like this possible. Everything we've accomplished here started with them.

Finally, we acknowledge the broader open-source communities behind the Eclipson Model T airframe, the INAV flight controller firmware, and the Arduino ecosystem. This project stands on the shoulders of thousands of engineers and hobbyists who shared their work freely. We intend to do the same.

References

[1] Eclipson. (2023). Model T — 3D Printed RC Airplane. Retrieved from https://www.eclipson-airplanes.com/model-t

[2] INAV Project. (2024). INAV — Navigation-enabled flight controller firmware. GitHub. Retrieved from https://github.com/iNavFlight/inav

[3] Matek Systems. (2023). F405-WMN Flight Controller Manual. Retrieved from http://www.mateksys.com/?portfolio=f405-wmn

[4] Arduino. (2024). Arduino Nano — Product Page and Technical Documentation. Retrieved from https://www.arduino.cc/en/pmwiki.php?n=Main/ArduinoBoardNano

[5] Adafruit Industries. (2024). Adafruit BME280 Library for Arduino. GitHub. Retrieved from https://github.com/adafruit/Adafruit_BME280_Library

[6] American Lung Association. (2025). State of the Air 2025 Report: San Antonio–New Braunfels–Kerrville, TX Metro Area. Retrieved from https://www.lung.org/research/sota

[7] SunnySky. (2023). X2212 Series Brushless Motor Specifications. Retrieved from https://www.sunnyskyusa.com

[8] HappyModel. (2024). EP1/EP2 ExpressLRS Nano Receiver Documentation. Retrieved from https://www.happymodel.cn

[9] Bosch Sensortec. (2023). BMP280 Digital Pressure Sensor — Datasheet. Retrieved from https://www.bosch-sensortec.com/products/environmental-sensors/pressure-sensors/bmp280/

[10] Zhengzhou Winsen Electronics. (2023). MQ-135 Sensitive Material Datasheet. Retrieved from https://www.winsen-sensor.com/sensors/air-quality-sensor/mq135.html

Appendix A: Parts & Budget

The following table lists all components purchased for the project, including substitutions made during the build phase. Items marked with an asterisk (*) replaced original components from the initial parts list.

Category	Component	Description	Price
Airframe	Filament *	OVERTURE Lightweight PLA — replaced eSUN LW PLA	$45
Airframe	Reinforcement	2mm Carbon Fiber Wing Spars (10pcs)	$12
Flight Systems	Flight Controller	Matek F405-WMN STM32F405 OSD Blackbox / INAV	$65
Flight Systems	GPS Module	BN-220 Dual GPS TTL	$23
Flight Systems	Motor *	SunnySky X2212 1250KV — replaced FLASH HOBBY D2830	$20
Flight Systems	ESC	40A Brushless ESC 2-4S 5V 3A BEC	$13
Flight Systems	Propeller *	Gemfan Nylon Propeller — replaced folding 1060/1160	$12
Flight Systems	Servos	RGBZONE ES08MA II Metal Gear Servos (4pcs)	$24
Flight Systems	Transmitter	Jumper Smart ELRS 2.4GHz RC Transmitter	$65
Flight Systems	Receiver	HappyModel ELRS EP Nano Long Range Receiver	$24
Research & Power	Microcontroller	Arduino Nano V3.0 ATmega328P	$8
Research & Power	Sensor Package *	BMP280 Atmospheric + MQ135 Air Quality	$18
Research & Power	Data Storage	SanDisk 32GB MicroSD & Card Reader Module	$21
Research & Power	Flight Battery	OVONIC 4S LiPo 2200mAh 14.8V 130C	$45
Research & Power	Charger	B6 Balance Charger 80W Mini LiPo	$30
Tools	Soldering Station	80W Electric Soldering Iron Adjustable Temp	$24
Tools	Wiring/Misc	Wire, XT60, Epoxy, Foam Cure, Insta-Set, Component Kit	$107
Tools	Toggle Switch	SPST Mini Toggle Switch 2 Pin (10pcs)	$8
TOTAL — All Purchased Components			**$465**

** Denotes component replaced from original parts list.*

Appendix B: Project Timeline

This project ran for approximately 28 weeks, beginning September 11, 2025 and targeting a first flight in March 2026. The timeline below reflects the actual phased progression of work across all four teams.

Weeks	Dates	Phase	Key Activities
1–4	Sep 11 – Oct 9, 2025	**Design & Planning**	Platform selection, parts research, team formation, quadcopter-to-fixed-wing transition, parts list finalized
5–8	Oct 10 – Nov 6, 2025	**Procurement & Setup**	Parts ordered, 3D printer calibration, filament testing, INAV research, Arduino sensor library testing
9–14	Nov 7 – Dec 18, 2025	**Fabrication Phase 1**	Eclipson Model T airframe printing begins, carbon fiber spar integration, fuselage assembly
15–18	Dec 19, 2025 – Jan 15, 2026	**Winter Break / Build**	Continued printing, wing assembly, component fitting, parts substitutions finalized
19–22	Jan 16 – Feb 12, 2026	**Electronics Integration**	ESC wiring, FC mounting, servo installation, ELRS binding, GPS wiring
23–25	Feb 13 – Mar 5, 2026	**Software & Sensor Config**	INAV configuration, Arduino sensor code, SD logging validation, bench testing
26–27	Mar 6 – Mar 19, 2026	**Pre-Assembly & Final Check**	Component verification, control surface linkage prep, center of gravity planning, pre-assembly inspection
28	Mar 22, 2026	**Physical Assembly Begins**	First day of full physical assembly — all procured components brought together for integration

Appendix B2: Team Roles & Responsibilities

The following table details each team member's role and primary responsibilities within the project.

Name	Role	Team	Primary Responsibilities
Aiden Ware Bosanko	Lead Researcher	Mechanical	Airframe design, 3D printing, structural assembly
William Van Cleave	Lead Researcher	Mechanical	Mechanical team lead, airframe assembly and structural integration
Kaleb Harris	Lead Researcher	Mechanical & Software	Airframe assembly, INAV configuration, data pipeline
Jorge Cruz	Lead Researcher	Mechanical, Software & Research Lead	Airframe assembly, INAV setup, software integration, and lead of research direction
Isaiah Jimenez	Lead Researcher	Electrical	Wiring, soldering, ESC & power system setup
Angelo Mejia	Lead Researcher	Electrical	Wiring, component integration, electrical validation
Samuel Cedeno	Research Contributor	Research	Research contributions and project support
Kyle McCracken	Research Contributor	Research	Ideation, conceptual input, and project support
Alex	Research Contributor	Electrical	Electrical assembly and wiring support
Austin	Research Contributor	Electrical	Electrical assembly and wiring support
Tyler Rankins	Research Contributor	Electrical	Electrical assembly and wiring support
Navora Medina	Support Contributor	Historian	Project historian — documentation, timeline tracking, and keeping the team on schedule
Talan	Support Contributor	Software	Arduino code development and sensor testing
Chris	Support Contributor	Software	Arduino code development and data logging

Appendix C: System Wiring Diagram

The diagram below illustrates the complete wiring and signal architecture of the UAV. The system is divided into two independent subsystems: the flight-critical avionics stack (powered directly from the LiPo battery via the ESC's BEC) and the research payload (managed by the Arduino Nano). This separation ensures that sensor activity cannot affect flight controller performance.

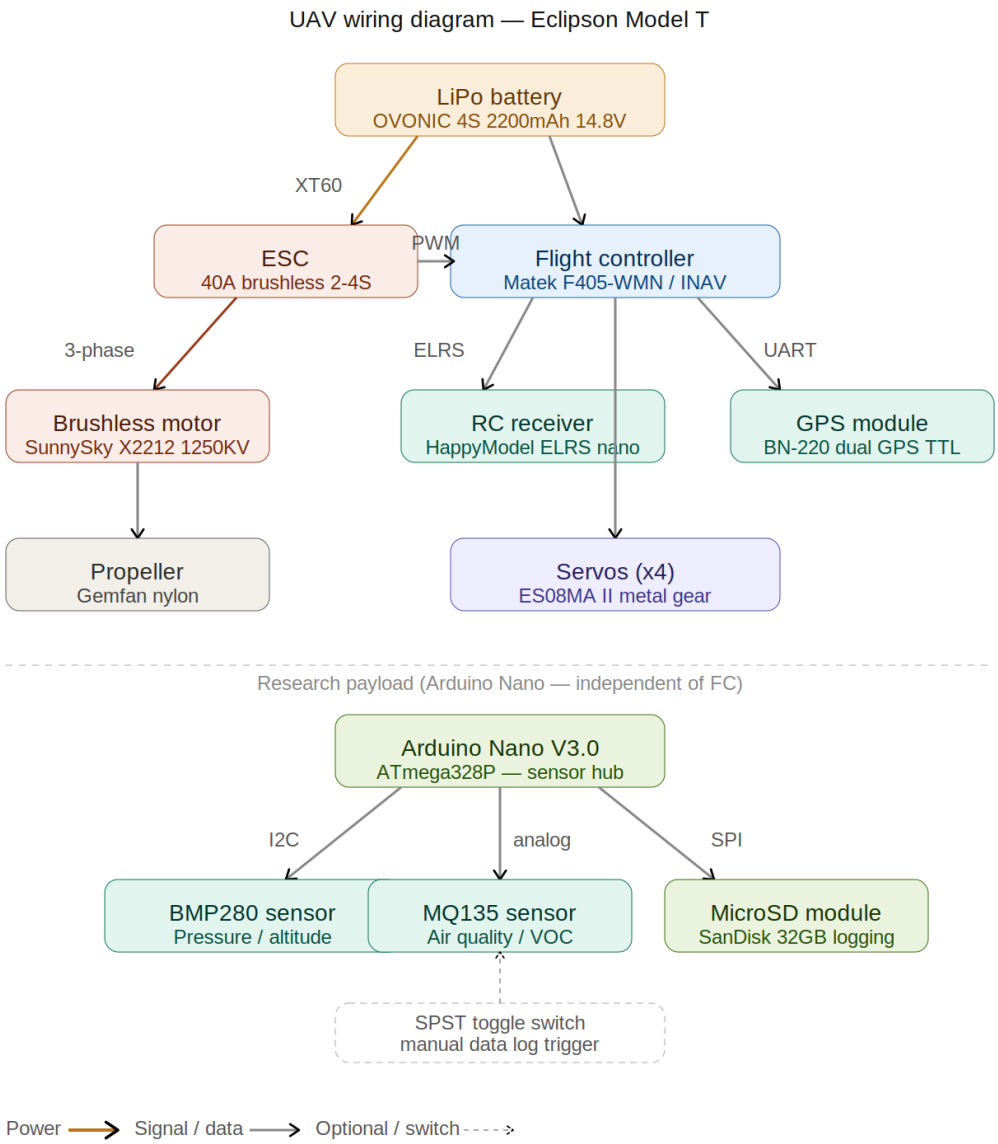
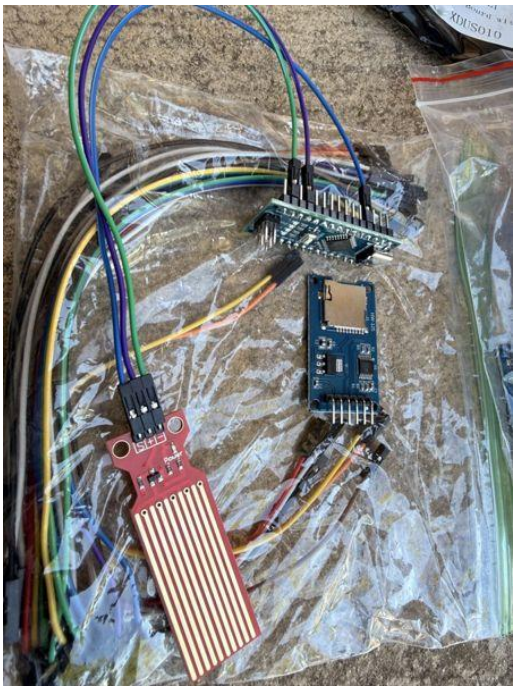


Figure C1: UAV system wiring diagram showing power flow (amber), signal/data connections (gray), and optional switch connections (dashed). The dashed line separates the flight systems (above) from the research payload (below).

Appendix D: Photos

The following photographs document the build process and current assembly status of the UAV as of March 2026.



Appendix E: Arduino Sensor Logger Code

The following is the intended Arduino Nano sensor logging code for the UAV's research payload. This code reads data from the BME280 atmospheric sensor and MQ135 air quality sensor, then logs all readings to a CSV file on the onboard microSD card at a fixed 2-second interval. Note: this code has not yet been fully validated in the integrated hardware environment and may require adjustments following bench testing.

Libraries Required:

- Adafruit BME280 Library
- Adafruit Unified Sensor Library
- SD (built-in Arduino library)
- SPI (built-in Arduino library)
- Wire (built-in Arduino library)

Pin Assignments:

- MQ135 Air Quality Sensor: Analog pin A1
- BME280 Atmospheric Sensor: I2C — SDA to A4, SCL to A5, address 0x76
- MicroSD Card Module: SPI — CS to D10, MOSI to D11, MISO to D12, SCK to D13

CSV Output Format:

Each row logged to data.csv contains: timestamp_ms, mq135_raw, mq135_percent, bme_temp_c, bme_humidity_percent, bme_pressure_hpa, bme_altitude_m

Source Code (nano_sensor_logger.ino):

```
#include <SPI.h>
#include <SD.h>
#include <math.h>
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME280.h>

// Pin assignments (Arduino Nano)
const uint8_t PIN_MQ135       = A1;
const uint8_t PIN_SD_CS       = 10;
const uint8_t BME280_I2C_ADDRESS = 0x76;

const char *LOG_FILE          = "data.csv";
const unsigned long LOG_INTERVAL_MS = 2000;

unsigned long lastLogMs = 0;
bool sdReady = false;
bool bmeReady = false;
Adafruit_BME280 bme;

struct SensorData {
  unsigned long timestampMs;
  int mq135Raw;
  float mq135Percent;
  float bmeTempC;
  float bmeHumidity;
  float bmePressurehPa;
  float bmeAltitudeM;
};

int readAveragedAnalog(uint8_t pin, uint8_t samples = 10) {
```

```
  long total = 0;
  for (uint8_t i = 0; i < samples; i++) {
    total += analogRead(pin);
    delay(3);
  }
  return (int)(total / samples);
}

SensorData readSensors() {
  SensorData d;
  d.timestampMs    = millis();
  d.mq135Raw       = readAveragedAnalog(PIN_MQ135);
  d.mq135Percent   = (d.mq135Raw / 1023.0) * 100.0;
  if (bmeReady) {
    d.bmeTempC       = bme.readTemperature();
    d.bmeHumidity    = bme.readHumidity();
    d.bmePressurehPa = bme.readPressure() / 100.0;
    d.bmeAltitudeM   = bme.readAltitude(1013.25);
  } else {
    d.bmeTempC = d.bmeHumidity = d.bmePressurehPa = d.bmeAltitudeM = NAN;
  }
  return d;
}

bool appendCsvRow(const SensorData &d) {
  if (!sdReady) return false;
  File file = SD.open(LOG_FILE, FILE_WRITE);
  if (!file) return false;
  file.print(d.timestampMs);   file.print(',');
  file.print(d.mq135Raw);      file.print(',');
  file.print(d.mq135Percent, 1); file.print(',');
  if (isnan(d.bmeTempC)) {
    file.print(F("nan,nan,nan,nan"));
  } else {
    file.print(d.bmeTempC, 2);       file.print(',');
    file.print(d.bmeHumidity, 2);    file.print(',');
    file.print(d.bmePressurehPa, 2); file.print(',');
    file.print(d.bmeAltitudeM, 2);
  }
  file.println();
  file.close();
  return true;
}

void setup() {
  Serial.begin(9600);
  Wire.begin();
  bmeReady = bme.begin(BME280_I2C_ADDRESS);
  sdReady  = SD.begin(PIN_SD_CS);
  if (sdReady) {
    File f = SD.open(LOG_FILE, FILE_WRITE);
    if (f && f.size() == 0)

f.println(F("timestamp_ms,mq135_raw,mq135_pct,bme_temp_c,bme_hum_pct,bme_pres_hpa,
bme_alt_m"));
    if (f) f.close();
  }
}

void loop() {
  unsigned long now = millis();
  if (now - lastLogMs >= LOG_INTERVAL_MS) {
```

```
        lastLogMs = now;
        SensorData data = readSensors();
        appendCsvRow(data);
      }
    }
```

Note: Full source file including serial debug output and water sensor support is available in the project repository. This appendix shows the core logging logic.